Dependência Funcional

Banco de Dados

Objetivos da aula

Ao final desta aula, você será capaz de:

- Compreender o conceito de dependência funcional
- Identificar dependências em tabelas
- Analisar problemas de redundância
- Detectar dependências em estruturas mal projetadas

Revisão rápida

Em um banco relacional, os dados são organizados em tabelas.

Cada linha representa um registro.

Cada coluna representa um atributo.

Pergunta-chave: quais atributos determinam outros atributos?

Conceito de dependência funcional

Uma dependência funcional ocorre quando o valor de um atributo (ou conjunto de atributos)
determina o valor de outro atributo.

Representação:

$$X \rightarrow Y$$

Leitura: "X determina Y".

Interpretando $X \to Y$

Se duas linhas possuem o mesmo valor em X,
elas devem possuir o mesmo valor em Y.

Se isso não acontece, a dependência **não** vale para a tabela.

Analogia: CPF e pessoa

Pense no **CPF** como o atributo determinante.

- Se duas fichas têm o mesmo CPF, elas devem apontar para a mesma pessoa.
- Então: $CPF \to Nome, DataNascimento.$

Agora o contrário não vale:

- `Nome -> CPF` é falso, porque podem existir pessoas com mesmo nome.

Resumo da analogia:

- atributo único e estável costuma determinar outros dados
- atributo ambíguo raramente determina algo com confiança

Exemplo 1: dependência válida

Tabela `Aluno`:

RA	Nome	Curso
101	Ana	ADS
102	Bruno	SI
103	Carla	ADS

Dependências observadas:

- $RA \rightarrow Nome$
- $RA \rightarrow Curso$

Exemplo 2: dependência inválida

Ainda na tabela `Aluno`:

RA	Nome	Curso
101	Ana	ADS
104	Ana	Medicina

`Nome \to Curso` é válida?

Não. O mesmo `Nome` aparece com cursos diferentes.

Chave candidata e dependência

Uma **chave candidata** determina todos os atributos da tabela.

Se `RA` for chave candidata em `Aluno`, então:

$$RA \rightarrow Nome, Curso, DataNascimento, Email$$

Dependência funcional total e parcial

Considere chave composta `(PedidoID, ProdutoID)`.

- **Total:** depende da chave toda.
- **Parcial:** depende de apenas parte da chave.

Exemplo:

- $(PedidoID, ProdutoID) \rightarrow Quantidade$ (total)
- $ProdutoID \rightarrow NomeProduto$ (parcial em relação à chave composta)

Dependência transitiva

Quando:

$$X \to Y \quad e \quad Y \to Z$$

Então existe dependência transitiva de X para Z.

Exemplo:

- `RA -> CodCurso`
- `CodCurso -> NomeCurso`
- logo `RA -> NomeCurso` (transitiva)

Redundância: por que é um problema?

Quando a tabela mistura entidades, o mesmo dado se repete várias vezes.

Consequências:

- aumento de armazenamento
- maior chance de inconsistências
- manutenção mais cara

Tabela mal projetada (exemplo)

Tabela `MatriculaGeral`:

RA	NomeAluno	CodCurso	NomeCurso	CoordCurso
101	Ana	ADS	Análise e Desenvolvimento de Sistemas	Prof. Livia
103	Carla	ADS	Análise e Desenvolvimento de Sistemas	Prof. Livia

Repare a repetição de `NomeCurso` e `CoordCurso`.

Dependências detectadas no exemplo

Na tabela `MatriculaGeral`, podemos observar:

- $RA \rightarrow NomeAluno, CodCurso$
- $CodCurso \rightarrow NomeCurso, CoordCurso$

Logo, há dependência transitiva:

- $RA \rightarrow NomeCurso, CoordCurso$

Isso indica necessidade de normalização.

Anomalias causadas pela redundância

1. **Anomalia de atualização**
    - alterar coordenador de ADS exige várias linhas
2. **Anomalia de inserção**
    - não consigo cadastrar curso sem aluno matriculado
3. **Anomalia de exclusão**
    - ao remover último aluno, perco dados do curso

Como detectar estruturas mal projetadas

Checklist prático:

1. Identifique a chave primária.
2. Liste dependências funcionais observadas.
3. Verifique dependências parciais.
4. Verifique dependências transitivas.
5. Procure atributos repetidos em muitas linhas.

Se houver parciais/transitivas, a modelagem pode estar fraca.

Exercício guiado

Tabela `Venda`:

VendaID	ClienteID	NomeCliente	CidadeCliente	ProdutoID	NomeProdut
9001	10	Maria	Cascavel	501	Mouse
9002	10	Maria	Cascavel	502	Teclado

Tarefa:

1. Liste as dependências funcionais.
2. Aponte redundâncias.
3. Sugira separação em tabelas.

Possível solução do exercício

Dependências prováveis:

- $ClienteID \rightarrow NomeCliente, CidadeCliente$
- $ProdutoID \rightarrow NomeProduto$
- $VendaID \rightarrow ClienteID$

Separação sugerida:

- `Cliente(ClienteID, NomeCliente, CidadeCliente)`
- `Produto(ProdutoID, NomeProduto)`
- `Venda(VendaID, ClienteID)`
- `ItemVenda(VendaID, ProdutoID, Quantidade)`

Certo x Errado 1: `RA \to Nome`

Certo (dependência válida):

RA	Nome
101	Ana
102	Bruno
101	Ana

Mesmo RA, mesmo Nome.

Errado (dependência inválida):

RA	Nome
101	Ana
101	Ana Lua

Certo x Errado 2: `ProdutoID \to NomeProduto`

Certo:

ProdutoID	NomeProduto
501	Mouse
502	Teclado
501	Mouse

Errado:

ProdutoID	NomeProduto
501	Mouse
501	Mouse Gamer RGB

Certo x Errado 3: chave composta

Considere a chave `(PedidoID, ProdutoID)` em `ItemPedido`.

Certo:

- $(PedidoID, ProdutoID) \rightarrow Quantidade, ValorUnitario$

Errado (dependência parcial em tabela errada):

- $ProdutoID \rightarrow ValorUnitario$ dentro de uma tabela histórica de pedido

Se o preço muda no tempo, guardar só por `ProdutoID` pode distorcer vendas antigas.

Mini estudo de caso de uso

Contexto: clínica médica quer registrar consultas.

Tabela inicial (`ConsultaGeral`):

ConsultaID	PacienteCPF	NomePaciente	MedicoCRM	NomeMedico	Espe
7001	111	Julia	900	Dr. Paulo	Card
7002	111	Julia	901	Dra. Lais	Clíni

Mini estudo de caso: análise

Dependências observadas:

- $PacienteCPF \rightarrow NomePaciente$
- $MedicoCRM \rightarrow NomeMedico, Especialidade$
- $ConsultaID \rightarrow PacienteCPF, MedicoCRM, DataConsulta$

Problemas:

- nome do paciente repetido em cada consulta
- dados do medico repetidos em cada consulta
- alto risco de inconsistência em atualizações

Mini estudo de caso: solução

Separação em tabelas:

- `Paciente(PacienteCPF, NomePaciente)`
- `Medico(MedicoCRM, NomeMedico, Especialidade)`
- `Consulta(ConsultaID, PacienteCPF, MedicoCRM, DataConsulta)`

Ganhos práticos:

1. menos redundância
2. atualização consistente
3. modelo pronto para normalização (até 3FN)

Resumo

- Dependência funcional expressa determinação entre atributos.
- Ajuda a encontrar problemas de projeto.
- Redundância gera anomalias.
- Detectar dependências parciais e transitivas é essencial para normalização.

Fechamento

Próxima etapa natural: aplicar essas ideias nas Formas Normais (1FN, 2FN, 3FN).

Perguntas?